Professional Guide to

AI-Powered Real-World Projects

Project 17 of 20

Full-Stack Architecture, Product Design, AI Integration, Testing, Deployment, and Viva Preparation

Purpose of This Guide

This document helps students move from a project title to a production-minded implementation. Every chapter defines the problem, users, scope, workflows, architecture, database, APIs, AI boundaries, security, tests, milestones, deliverables, and evaluation criteria.

Student Project Development Edition

Academic Year 2026

How to Use This Book

Choose one project only after reading its problem definition, minimum viable product, risks, and semester plan. Start with deterministic business rules and a reliable full-stack workflow before adding AI features. Treat every AI output as untrusted input that must be validated, stored with provenance, and reviewed when the decision can affect a person. Use the chapter checklists during weekly reviews and include evidence such as diagrams, API tests, screenshots, logs, and deployment links in the final submission.

Recommended Team Responsibilities

Responsibility	Expected Ownership
Product Lead	Problem research, scope, user stories, acceptance criteria, demo narrative, and stakeholder communication.
Frontend Lead	Design system, responsive screens, accessibility, client state, API integration, and usability testing.
Backend Lead	Domain model, APIs, authorization, validation, jobs, integrations, logging, and deployment readiness.
Data and AI Lead	Datasets, evaluation, prompts or models, safety checks, fallbacks, monitoring, and AI documentation.
Quality Lead	Test plan, defect tracking, security checks, performance evidence, release checklist, and documentation quality.

Definition of a Professional Student Project

- The product solves a specific problem for clearly defined users.
- The core workflow works end to end with persistent data and authorization.
- The interface handles loading, empty, success, validation, and failure states.
- The backend validates all input and enforces permissions independently of the frontend.
- AI is used only where it creates measurable value and has a deterministic fallback.
- The team can explain architecture choices, trade-offs, limitations, and future work.
- Tests, logs, monitoring, deployment, and documentation are treated as product features.

Contents

How to Use This Book **i**
Recommended Team Responsibilities i
Definition of a Professional Student Project i

17 AI Career Roadmap Generator **1**
17.1 Why This Project Matters 1
17.1.1 Real-World Context 1
17.1.2 Value Proposition 1
17.2 Problem Definition and Research Plan 2
17.2.1 Suggested Discovery Activities 2
17.3 Scope and Product Boundaries 2
17.3.1 Minimum Viable Product 2
17.3.2 Out of Scope for the First Release 3
17.4 Users, Roles, and Permissions 3
17.4.1 Authorization Rules 3
17.5 Functional Requirements 3
17.5.1 FR-01: Career Profile 4
17.5.2 FR-02: Goal Selection 4
17.5.3 FR-03: Skill Inventory 4
17.5.4 FR-04: Gap Analysis 4
17.5.5 FR-05: Roadmap Builder 5
17.5.6 FR-06: Progress Tracking 5
17.5.7 FR-07: Project Planner 5
17.5.8 FR-08: Mentor Review 6
17.6 Non-Functional Requirements 6
17.6.1 Suggested Service Targets 7
17.7 End-to-End User Workflows 7
17.7.1 Primary Success Flow 7
17.7.2 Exception Flows 7
17.8 Information Architecture and Screen Plan 8
17.8.1 Frontend State Checklist 8
17.9 Recommended Technical Architecture 8
17.9.1 Architecture Style 9
17.9.2 Suggested Technology Stack 9
17.9.3 Logical Component Flow 9
17.10 Frontend Engineering Blueprint 9
17.10.1 Suggested Folder Structure 10

17.10.2 Frontend Implementation Rules . . . 10
17.11 Backend Engineering Blueprint . . . 10
17.11.1 Suggested Folder Structure . . . 10
17.11.2 Backend Rules . . . 11
17.12 Database Design . . . 11
17.12.1 Common Record Fields . . . 12
17.12.2 Indexing Plan . . . 12
17.13 API Design . . . 13
17.13.1 API Response Standard . . . 13
17.14 Scoring, Matching, or Decision Logic . . . 14
17.15 AI Integration Plan . . . 14
17.15.1 Useful AI Capabilities . . . 14
17.15.2 Responsible AI Pipeline . . . 14
17.15.3 AI Evaluation Dataset . . . 15
17.16 Security, Privacy, and Ethics . . . 15
17.16.1 Project-Specific Risk Register . . . 16
17.17 Analytics and Success Measurement . . . 16
17.17.1 Analytics Events . . . 17
17.18 Testing Strategy . . . 17
17.18.1 TC-01: authentication and session handling . . . 18
17.18.2 TC-02: role-based authorization . . . 18
17.18.3 TC-03: request payload validation . . . 18
17.18.4 TC-04: ownership enforcement . . . 18
17.18.5 TC-05: duplicate submission handling . . . 18
17.18.6 TC-06: search and filter correctness . . . 18
17.18.7 TC-07: pagination and stable sorting . . . 19
17.18.8 TC-08: file upload validation . . . 19
17.18.9 TC-09: notification delivery . . . 19
17.18.10 TC-10: background job retry behavior . . . 19
17.18.11 TC-11: AI timeout and fallback behavior . . . 19
17.18.12 TC-12: AI structured-output validation . . . 20
17.18.13 TC-13: audit event creation . . . 20
17.18.14 TC-14: privacy redaction . . . 20
17.18.15 TC-15: accessibility keyboard flow . . . 20
17.18.16 TC-16: responsive layout . . . 20
17.18.17 TC-17: empty and loading states . . . 20
17.18.18 TC-18: error recovery . . . 21
17.18.19 TC-19: database index usage . . . 21
17.18.20 TC-20: backup restoration . . . 21
17.18.21 Required Test Layers . . . 21
17.19 Detailed Product Backlog . . . 21
17.19.1 US-01: account access . . . 22
17.19.2 US-02: profile completion . . . 22
17.19.3 US-03: goal selection . . . 23
17.19.4 US-04: skill inventory . . . 23
17.19.5 US-05: gap analysis . . . 24
17.19.6 US-06: roadmap builder . . . 24

17.19.7 US-07: progress tracking . . . 25
17.19.8 US-08: project planner . . . 25
17.19.9 US-09: notifications . . . 26
17.19.10 US-10: search and filters . . . 26
17.19.11 US-11: reports . . . 27
17.19.12 US-12: administration . . . 27
17.20 Implementation Roadmap . . . 28
17.20.1 Phase 0: Discovery . . . 28
17.20.2 Phase 1: Foundation . . . 28
17.20.3 Phase 2: Identity . . . 28
17.20.4 Phase 3: Core Records . . . 29
17.20.5 Phase 4: Collaboration . . . 29
17.20.6 Phase 5: Intelligence . . . 29
17.20.7 Phase 6: Analytics . . . 29
17.20.8 Phase 7: Hardening . . . 30
17.20.9 Phase 8: Pilot . . . 30
17.20.10 Phase 9: Production . . . 30
17.21 Semester Execution Plan . . . 30
17.22 Deployment and Operations . . . 31
17.22.1 Environment Variables . . . 31
17.23 Documentation Deliverables . . . 31
17.24 Demonstration Script . . . 32
17.25 Viva and Interview Preparation . . . 32
17.26 Evaluation Rubric . . . 33
17.27 Project Completion Checklist . . . 33
17.28 Conclusion . . . 34

Cross-Project Architecture Checklist **35**

Final Advice to Student Teams **36**

Project 17

AI Career Roadmap Generator

Executive Summary: ACRG

Domain: Personalized career planning and skill development.
Problem: Students consume disconnected learning resources without a realistic sequence that reflects their current skills, time, goals, and evidence.
Proposed product: A career planning assistant that creates an editable, evidence-based roadmap with milestones, projects, assessments, and progress reviews.

17.1 Why This Project Matters

This project is suitable for a major academic submission because it combines product thinking, user experience, backend engineering, data modeling, security, analytics, and responsible AI. A strong implementation should demonstrate one complete operational journey instead of presenting disconnected screens or static dashboard values. The project must be evaluated by the quality of its decisions, evidence, and user outcomes rather than by the number of libraries included.

17.1.1 Real-World Context

Students consume disconnected learning resources without a realistic sequence that reflects their current skills, time, goals, and evidence. Existing informal processes usually depend on spreadsheets, chat groups, manual follow-up, or personal networks. Those approaches are difficult to search, difficult to audit, and difficult to scale across a department or institution. A career planning assistant that creates an editable, evidence-based roadmap with milestones, projects, assessments, and progress reviews.

17.1.2 Value Proposition

- For end users, the platform reduces confusion and makes personalized career planning and skill development workflows easier to complete.
- For operational teams, the platform creates consistent records, queues, decisions, and reports.
- For administrators, the platform provides measurable activity and outcome indicators without relying on manual consolidation.

- For students building the system, the project demonstrates end-to-end engineering and defensible product judgment.

17.2 Problem Definition and Research Plan

Research Area	Questions to Answer Before Development
User behavior	How do people currently handle personalized career planning and skill development, and where do they lose the most time or trust?
Current tools	Which spreadsheets, forms, chat groups, portals, or manual approvals are used today?
Decision rules	Which decisions are objective, which require human judgment, and which must never be delegated to AI?
Data availability	Which fields are already available, who owns them, and how accurate or sensitive are they?
Success	Which behavior proves that AI Career Roadmap Generator is useful after four, eight, and twelve weeks?
Adoption	What would prevent a user from completing the main workflow, and what trust signals are required?

17.2.1 Suggested Discovery Activities

1. Interview at least five target users from two different roles.
2. Observe one current workflow from beginning to end.
3. Create a problem map that separates symptoms from root causes.
4. Write the top ten assumptions and mark each as verified, uncertain, or false.
5. Create a low-fidelity prototype and test it before designing the final interface.
6. Define baseline measurements so improvement can be demonstrated after the pilot.

17.3 Scope and Product Boundaries

17.3.1 Minimum Viable Product

- Secure access for Student, Career Mentor, Placement Officer, Career Content Administrator.
- A complete workflow across Career Profile, Goal Selection, Skill Inventory, Gap Analysis, Roadmap Builder.
- Persistent records with ownership, timestamps, validation, and lifecycle status.
- A role-specific dashboard driven by real backend data.
- At least one useful intelligence feature: career pathway generation.

- An administrator workflow for configuration, review, audit, or support.
- A deployed demonstration environment with seeded sample data.

17.3.2 Out of Scope for the First Release

- Native mobile applications unless the team already has a stable responsive web version.
- Microservices before the modular monolith has proven performance or ownership limits.
- Fully autonomous high-impact decisions based only on AI output.
- Complex payment settlement, biometric surveillance, or legal identity verification without institutional support.
- Large third-party integrations that do not improve the primary demonstration workflow.
- Features that cannot be tested with available users, data, time, or infrastructure.

17.4 Users, Roles, and Permissions

Role	Core Responsibilities
Student	register and complete a verified profile; view a personalized dashboard; and receive status and deadline notifications.
Career Mentor	sign in securely and recover account access; create and update owned records; and review an activity history.
Placement Officer	view a personalized dashboard; search and filter permitted records; and export or share an authorized report.
Career Content Administrator	create and update owned records; receive status and deadline notifications; and register and complete a verified profile.

17.4.1 Authorization Rules

- A user can read or modify a private record only when ownership or an explicit role grant permits it.
- Administrative access must be checked on the server and recorded in an audit trail.
- Public pages must expose an allowlisted view model rather than the full database object.
- File access must use signed or permission-checked URLs instead of permanent public links.
- Role changes, verification decisions, exports, and destructive actions require audit events.
- Suspended users retain historical records but cannot create new activity.

17.5 Functional Requirements

17.5.1 FR-01: Career Profile

1. The system shall provide a clear entry point for the career profile workflow.
2. The backend shall validate every career profile request before changing persistent state.
3. The interface shall show loading, empty, success, validation, permission, and server-error states for career profile.
4. Authorized users shall be able to search, filter, sort, or review relevant career profile records.
5. Important career profile state changes shall create timestamps, actor references, and audit events.
6. The module shall publish domain events when another module needs notification or analytics updates.

17.5.2 FR-02: Goal Selection

1. The system shall provide a clear entry point for the goal selection workflow.
2. The backend shall validate every goal selection request before changing persistent state.
3. The interface shall show loading, empty, success, validation, permission, and server-error states for goal selection.
4. Authorized users shall be able to search, filter, sort, or review relevant goal selection records.
5. Important goal selection state changes shall create timestamps, actor references, and audit events.
6. The module shall publish domain events when another module needs notification or analytics updates.

17.5.3 FR-03: Skill Inventory

1. The system shall provide a clear entry point for the skill inventory workflow.
2. The backend shall validate every skill inventory request before changing persistent state.
3. The interface shall show loading, empty, success, validation, permission, and server-error states for skill inventory.
4. Authorized users shall be able to search, filter, sort, or review relevant skill inventory records.
5. Important skill inventory state changes shall create timestamps, actor references, and audit events.
6. The module shall publish domain events when another module needs notification or analytics updates.

17.5.4 FR-04: Gap Analysis

1. The system shall provide a clear entry point for the gap analysis workflow.
2. The backend shall validate every gap analysis request before changing persistent state.

3. The interface shall show loading, empty, success, validation, permission, and server-error states for gap analysis.
4. Authorized users shall be able to search, filter, sort, or review relevant gap analysis records.
5. Important gap analysis state changes shall create timestamps, actor references, and audit events.
6. The module shall publish domain events when another module needs notification or analytics updates.

17.5.5 FR-05: Roadmap Builder

1. The system shall provide a clear entry point for the roadmap builder workflow.
2. The backend shall validate every roadmap builder request before changing persistent state.
3. The interface shall show loading, empty, success, validation, permission, and server-error states for roadmap builder.
4. Authorized users shall be able to search, filter, sort, or review relevant roadmap builder records.
5. Important roadmap builder state changes shall create timestamps, actor references, and audit events.
6. The module shall publish domain events when another module needs notification or analytics updates.

17.5.6 FR-06: Progress Tracking

1. The system shall provide a clear entry point for the progress tracking workflow.
2. The backend shall validate every progress tracking request before changing persistent state.
3. The interface shall show loading, empty, success, validation, permission, and server-error states for progress tracking.
4. Authorized users shall be able to search, filter, sort, or review relevant progress tracking records.
5. Important progress tracking state changes shall create timestamps, actor references, and audit events.
6. The module shall publish domain events when another module needs notification or analytics updates.

17.5.7 FR-07: Project Planner

1. The system shall provide a clear entry point for the project planner workflow.
2. The backend shall validate every project planner request before changing persistent state.
3. The interface shall show loading, empty, success, validation, permission, and server-error states for project planner.
4. Authorized users shall be able to search, filter, sort, or review relevant project planner records.

5. Important project planner state changes shall create timestamps, actor references, and audit events.
6. The module shall publish domain events when another module needs notification or analytics updates.

17.5.8 FR-08: Mentor Review

1. The system shall provide a clear entry point for the mentor review workflow.
2. The backend shall validate every mentor review request before changing persistent state.
3. The interface shall show loading, empty, success, validation, permission, and server-error states for mentor review.
4. Authorized users shall be able to search, filter, sort, or review relevant mentor review records.
5. Important mentor review state changes shall create timestamps, actor references, and audit events.
6. The module shall publish domain events when another module needs notification or analytics updates.

17.6 Non-Functional Requirements

Quality Attribute	Professional Requirement
Security	Protect identity, authorization boundaries, secrets, uploads, and sensitive records.
Privacy	Collect only necessary data and expose it only to explicitly permitted roles.
Performance	Keep common reads responsive and move expensive processing to background jobs.
Scalability	Support growth through stateless APIs, indexes, caching, queues, and object storage.
Reliability	Use validation, idempotency, retries, backups, health checks, and clear failure states.
Accessibility	Meet keyboard, contrast, semantic markup, focus, and screen-reader expectations.
Maintainability	Keep modules cohesive, interfaces documented, and business rules covered by tests.
Observability	Record structured logs, metrics, traces, audit events, and actionable alerts.

17.6.1 Suggested Service Targets

- Ninety-fifth percentile API response below 500 ms for indexed read operations in the pilot environment.
- User-facing write operations should acknowledge success or failure within two seconds unless intentionally asynchronous.
- Background AI or media jobs must expose queued, processing, completed, failed, and retrying states.
- The system should support at least 100 concurrent pilot users without data corruption or authorization leakage.
- Daily database backups and a documented restore exercise are required before the final demonstration.
- Critical accessibility flows must be usable with keyboard navigation and visible focus indicators.

17.7 End-to-End User Workflows

17.7.1 Primary Success Flow

1. A Student creates an account and completes the required profile fields.
2. The user starts the goal selection workflow and submits validated information.
3. The system creates or updates a CareerGoal record and records ownership.
4. The roadmap builder logic evaluates the record using transparent rules.
5. A Career Mentor reviews or responds when human action is required.
6. The system creates a GapAnalysis or equivalent outcome record.
7. The user receives a notification and sees the updated status on the dashboard.
8. Analytics update the selected measures, including roadmap activation, milestone completion, skill evidence growth.

17.7.2 Exception Flows

- Incomplete input is rejected with field-level guidance and no partial domain record.
- Duplicate submissions return an idempotent response or a clear conflict message.
- Unauthorized access returns a generic permission error and creates a security event when appropriate.
- Unavailable AI services trigger a retry or deterministic fallback without blocking the complete product.
- Rejected or disputed decisions preserve history and expose the permitted appeal or correction path.

- Failed notifications do not roll back the underlying business transaction.

17.8 Information Architecture and Screen Plan

Screen	Required Experience
Public Landing Page	Problem, benefits, trust signals, process explanation, and primary call to action.
Authentication	Registration, login, verification, password recovery, and secure session feedback.
Student Dashboard	Priority tasks, recent activity, roadmap activation, status summaries, and recommended next action.
Goal Selection Workspace	Create, edit, validate, save draft, submit, and review owned information.
Skill Inventory Screen	Focused workflow with contextual help, status visibility, and evidence display.
Roadmap Builder Results	Explain the result, source data, confidence, limitations, and available next steps.
Notifications Center	Unread state, category filters, deep links, and preference controls.
Career Content Administrator Console	Configuration, review queues, audit history, metrics, and support tools.

17.8.1 Frontend State Checklist

- Initial loading skeleton
- No-data onboarding state
- Successful data state
- Field validation state
- Permission-denied state
- Network or server error state
- Background processing state
- Partial data warning
- Confirmation for destructive actions
- Mobile navigation and small-screen tables

17.9 Recommended Technical Architecture

17.9.1 Architecture Style

Use a modular monolith for the first production-minded version. Keep domain modules independent inside one deployable backend, and separate long-running AI, media, or notification work through a queue. This approach is easier to test and deploy than premature microservices while still creating clear boundaries for future extraction.

17.9.2 Suggested Technology Stack

Layer	Recommended Choice
Web application	React with Vite or Next.js, TypeScript, Tailwind CSS, React Query, React Hook Form, and an accessible component system.
API	Node.js with Express or NestJS, TypeScript, Zod or Joi validation, structured errors, and OpenAPI documentation.
Database	PostgreSQL for relational integrity, or MongoDB when the team can justify document-shaped aggregates and indexes.
Cache and jobs	Redis for short-lived cache, rate limits, distributed locks, and a BullMQ-compatible background queue.
Files	S3-compatible object storage or Cloudinary with signed uploads, MIME validation, size limits, and retention rules.
AI boundary	A dedicated service module that validates structured output, records model metadata, and supports fallbacks.
Deployment	Docker containers, managed database, environment-based configuration, HTTPS, logs, metrics, and automated migrations.

17.9.3 Logical Component Flow

Listing 17.1: Logical request and processing flow

```
Browser / Mobile Web
       |
       v
API Gateway and Authentication Middleware
       |
       v
Domain Service -> Repository -> Primary Database
       |
       +--> Event / Job Queue --> Worker --> AI or External Service
       | |
       +<------ Status and Result ----+
       |
       +--> Notification Service
       +--> Analytics Event Store
       +--> Audit Log
```

17.10 Frontend Engineering Blueprint

17.10.1 Suggested Folder Structure

Listing 17.2: Feature-oriented frontend structure

```
frontend/
  src/
    app/
      router/
      providers/
      layouts/
    components/
      ui/
      feedback/
      navigation/
    features/
      acrg/
        api/
        components/
        hooks/
        pages/
        schemas/
        types/
    lib/
      http/
      auth/
      validation/
      analytics/
    styles/
    tests/
```

17.10.2 Frontend Implementation Rules

- Use server state tooling for API data and keep local UI state separate.
- Validate forms on the client for usability and again on the server for security.
- Centralize API errors into user-friendly messages while preserving request identifiers for debugging.
- Use route-level authorization only as a convenience; the backend remains the authority.
- Build reusable accessible primitives for buttons, inputs, dialogs, tables, status badges, and empty states.
- Measure major flows with privacy-conscious analytics events.
- Never expose API keys, database credentials, private prompts, or privileged service tokens in frontend code.

17.11 Backend Engineering Blueprint

17.11.1 Suggested Folder Structure

Listing 17.3: Modular backend structure

```
backend/
  src/
    config/
    common/
      auth/
      errors/
      logging/
      validation/
    modules/
      career-profile/
      goal-selection/
      skill-inventory/
      gap-analysis/
      roadmap-builder/
      progress-tracking/
      project-planner/
      mentor-review/
    jobs/
    integrations/
    database/
      migrations/
      seeds/
    app.ts
    server.ts
  tests/
  Dockerfile
```

17.11.2 Backend Rules

- Controllers translate HTTP input and output; domain services contain business rules.
- Repositories isolate database access and make query behavior testable.
- Validation schemas are shared by handlers and API documentation where practical.
- Business errors use stable machine-readable codes and safe user messages.
- Write endpoints accept idempotency keys when duplicate actions would be harmful.
- External integrations use timeouts, limited retries, circuit-breaking behavior, and request correlation identifiers.
- Secrets are loaded from environment or a secret manager and are never committed to Git.

17.12 Database Design

Entity	Purpose
User	Stores user information for career profile, including identifiers, lifecycle state, ownership, and timestamps.

Entity	Purpose
CareerProfile	Stores careerprofile information for goal selection, including identifiers, lifecycle state, ownership, and timestamps.
CareerGoal	Stores careergoal information for skill inventory, including identifiers, lifecycle state, ownership, and timestamps.
Skill	Stores skill information for gap analysis, including identifiers, lifecycle state, ownership, and timestamps.
SkillEvidence	Stores skillevidence information for roadmap builder, including identifiers, lifecycle state, ownership, and timestamps.
GapAnalysis	Stores gapanalysis information for progress tracking, including identifiers, lifecycle state, ownership, and timestamps.
Roadmap	Stores roadmap information for project planner, including identifiers, lifecycle state, ownership, and timestamps.
RoadmapMilestone	Stores roadmapmilestone information for mentor review, including identifiers, lifecycle state, ownership, and timestamps.
ProgressUpdate	Stores progressupdate information for career profile, including identifiers, lifecycle state, ownership, and timestamps.
MentorComment	Stores mentorcomment information for goal selection, including identifiers, lifecycle state, ownership, and timestamps.

17.12.1 Common Record Fields

- id: stable primary identifier
- organizationId or campusId: tenant boundary when multiple institutions are supported
- createdBy and updatedBy: actor references
- createdAt and updatedAt: server-generated timestamps
- status: controlled lifecycle state
- version: optimistic concurrency or revision number
- deletedAt: optional soft-delete timestamp for recoverable records
- metadata: limited extension object with documented keys

17.12.2 Indexing Plan

- Unique index on normalized email and other true business identifiers.
- Compound indexes for the most common tenant, status, owner, and date filters.
- Text or search-engine index only for fields intended for discovery.
- Time-based index for events, notifications, audits, and analytics rollups.

- Foreign-key or reference indexes for every frequently joined relation.
- A query plan check for dashboard and search endpoints before the final release.

17.13 API Design

Method	Endpoint	Responsibility
POST	/api/v1/acrg/auth/register	Create an account and required profile seed.
POST	/api/v1/acrg/auth/login	Create a secure authenticated session.
GET	/api/v1/acrg/me	Return the current user's safe profile view.
PATCH	/api/v1/acrg/me	Update allowlisted profile fields.
GET	/api/v1/acrg/careergoal	List authorized CareerGoal records with filters and pagination.
POST	/api/v1/acrg/careergoal	Create and validate a CareerGoal record.
GET	/api/v1/acrg/careergoal/:id	Read an authorized CareerGoal detail view.
PATCH	/api/v1/acrg/careergoal/:id	Update mutable CareerGoal fields.
POST	/api/v1/acrg/gapanalysis	Start the Roadmap Builder or review workflow.
GET	/api/v1/acrg/dashboard	Return role-specific summaries including roadmap activation.
GET	/api/v1/acrg/notifications	List user notifications using cursor pagination.
GET	/api/v1/acrg/admin/audit-events	Return permission-checked audit history.

17.13.1 API Response Standard

Listing 17.4: Example successful response envelope

```
{
  "data": { "id": "record-id", "status": "active" },
  "meta": { "requestId": "req-id", "timestamp": "ISO-8601" },
  "error": null
}
```

Listing 17.5: Example validation error envelope

```
{
  "data": null,
  "meta": { "requestId": "req-id", "timestamp": "ISO-8601" },
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Please correct the highlighted fields.",
    "details": [{ "field": "title", "reason": "required" }]
  }
}
```

17.14 Scoring, Matching, or Decision Logic

Begin with a deterministic and explainable baseline. A transparent baseline is easier to test, demonstrate, and improve than an opaque AI-only score. Store each component score and explanation so that users and reviewers can understand the result.

Component	Example Weight and Interpretation
Profile or input completeness	15 percent; enough reliable data exists to perform the workflow.
Core eligibility or relevance	30 percent; mandatory domain requirements are satisfied.
Evidence or quality strength	25 percent; claims are supported by recent and relevant evidence.
Compatibility or operational fit	20 percent; preferences, constraints, status, and availability align.
Trust and recency	10 percent; verified, recent, and non-disputed information receives greater confidence.

Scoring Rule

Weights are examples, not universal truth. The team must derive final weights from interviews, pilot evidence, and stakeholder review. High-impact decisions require a human-readable explanation and an appeal or correction path.

17.15 AI Integration Plan

17.15.1 Useful AI Capabilities

- career pathway generation: use a constrained input contract, structured output schema, and measurable evaluation set.
- skill sequencing: use a constrained input contract, structured output schema, and measurable evaluation set.
- project recommendation: use a constrained input contract, structured output schema, and measurable evaluation set.
- weekly plan generation: use a constrained input contract, structured output schema, and measurable evaluation set.
- roadmap revision: use a constrained input contract, structured output schema, and measurable evaluation set.

17.15.2 Responsible AI Pipeline

1. Confirm that the user has permission to process the selected data.

2. Remove unnecessary personal or secret information before sending data to a model.
3. Build a versioned prompt or model request with explicit task, context, constraints, and output schema.
4. Apply timeouts, rate limits, and cost controls.
5. Validate the response against a strict schema.
6. Reject or repair malformed output without trusting free-form text as executable instructions.
7. Apply domain rules and safety checks after model processing.
8. Store model name, request version, timestamp, confidence indicators, and reviewer corrections.
9. Display limitations and provide a non-AI fallback or manual review path.
10. Evaluate accuracy and usefulness on a representative test set before release.

17.15.3 AI Evaluation Dataset

- At least 50 anonymized or synthetic examples representing normal inputs.
- Edge cases with missing, contradictory, multilingual, or low-quality input.
- Adversarial text attempting prompt injection or policy bypass.
- Examples from different user groups to inspect uneven quality.
- Expected structured outputs reviewed by a domain-aware human.
- A failure log that distinguishes model errors, prompt errors, data errors, and product-rule errors.

17.16 Security, Privacy, and Ethics

Risk	Required Control
Account takeover	Strong password hashing, secure cookies or carefully protected tokens, rate limiting, verification, and session revocation.
Broken access control	Central authorization policies, ownership checks, tenant checks, and negative authorization tests.
Unsafe input	Schema validation, sanitization where needed, parameterized queries, and safe file processing.
Sensitive file exposure	Private object storage, signed URLs, malware scanning where available, and retention controls.
Data over-collection	Field-level purpose review, consent, minimization, deletion, and export procedures.
AI data leakage	Redaction, provider settings review, prompt isolation, output validation, and prohibited-data rules.

Risk	Required Control
Abuse and fraud	Reporting, moderation queues, velocity limits, evidence review, suspension, and appeals.
Audit gaps	Immutable or append-oriented security events with actor, action, target, time, and request identifier.

17.16.1 Project-Specific Risk Register

17.16.1.1 Risk 1: career advice may become outdated

- Define the likelihood and impact before implementation.
- Add a preventive product or technical control.
- Add a detection signal and responsible owner.
- Document the user-facing correction, appeal, or recovery path.
- Retest the risk during pilot review.

17.16.1.2 Risk 2: overly ambitious plans can discourage users

- Define the likelihood and impact before implementation.
- Add a preventive product or technical control.
- Add a detection signal and responsible owner.
- Document the user-facing correction, appeal, or recovery path.
- Retest the risk during pilot review.

17.16.1.3 Risk 3: course recommendations may appear promotional

- Define the likelihood and impact before implementation.
- Add a preventive product or technical control.
- Add a detection signal and responsible owner.
- Document the user-facing correction, appeal, or recovery path.
- Retest the risk during pilot review.

17.17 Analytics and Success Measurement

Metric	Definition and Use
roadmap activation	Define roadmap activation with an exact numerator, denominator, event source, reporting period, and owner. Use it to evaluate whether AI Career Roadmap Generator improves real behavior rather than only page views.
milestone completion	Define milestone completion with an exact numerator, denominator, event source, reporting period, and owner. Use it to evaluate whether AI Career Roadmap Generator improves real behavior rather than only page views.
skill evidence growth	Define skill evidence growth with an exact numerator, denominator, event source, reporting period, and owner. Use it to evaluate whether AI Career Roadmap Generator improves real behavior rather than only page views.
mentor approval	Define mentor approval with an exact numerator, denominator, event source, reporting period, and owner. Use it to evaluate whether AI Career Roadmap Generator improves real behavior rather than only page views.
career goal retention	Define career goal retention with an exact numerator, denominator, event source, reporting period, and owner. Use it to evaluate whether AI Career Roadmap Generator improves real behavior rather than only page views.

17.17.1 Analytics Events

- account_registered
- profile_completed
- goal_selection_started
- skill_inventory_submitted
- roadmap_builder_completed
- result_viewed
- recommendation_accepted
- notification_opened
- report_exported
- support_or_appeal_created

17.18 Testing Strategy

17.18.1 TC-01: authentication and session handling

- Create a positive test for authentication and session handling.
- Create a negative or unauthorized test for authentication and session handling.
- Verify persisted data and side effects after authentication and session handling.
- Record the expected API status, UI state, audit event, and notification behavior.

17.18.2 TC-02: role-based authorization

- Create a positive test for role-based authorization.
- Create a negative or unauthorized test for role-based authorization.
- Verify persisted data and side effects after role-based authorization.
- Record the expected API status, UI state, audit event, and notification behavior.

17.18.3 TC-03: request payload validation

- Create a positive test for request payload validation.
- Create a negative or unauthorized test for request payload validation.
- Verify persisted data and side effects after request payload validation.
- Record the expected API status, UI state, audit event, and notification behavior.

17.18.4 TC-04: ownership enforcement

- Create a positive test for ownership enforcement.
- Create a negative or unauthorized test for ownership enforcement.
- Verify persisted data and side effects after ownership enforcement.
- Record the expected API status, UI state, audit event, and notification behavior.

17.18.5 TC-05: duplicate submission handling

- Create a positive test for duplicate submission handling.
- Create a negative or unauthorized test for duplicate submission handling.
- Verify persisted data and side effects after duplicate submission handling.
- Record the expected API status, UI state, audit event, and notification behavior.

17.18.6 TC-06: search and filter correctness

- Create a positive test for search and filter correctness.
- Create a negative or unauthorized test for search and filter correctness.

- Verify persisted data and side effects after search and filter correctness.
- Record the expected API status, UI state, audit event, and notification behavior.

17.18.7 TC-07: pagination and stable sorting

- Create a positive test for pagination and stable sorting.
- Create a negative or unauthorized test for pagination and stable sorting.
- Verify persisted data and side effects after pagination and stable sorting.
- Record the expected API status, UI state, audit event, and notification behavior.

17.18.8 TC-08: file upload validation

- Create a positive test for file upload validation.
- Create a negative or unauthorized test for file upload validation.
- Verify persisted data and side effects after file upload validation.
- Record the expected API status, UI state, audit event, and notification behavior.

17.18.9 TC-09: notification delivery

- Create a positive test for notification delivery.
- Create a negative or unauthorized test for notification delivery.
- Verify persisted data and side effects after notification delivery.
- Record the expected API status, UI state, audit event, and notification behavior.

17.18.10 TC-10: background job retry behavior

- Create a positive test for background job retry behavior.
- Create a negative or unauthorized test for background job retry behavior.
- Verify persisted data and side effects after background job retry behavior.
- Record the expected API status, UI state, audit event, and notification behavior.

17.18.11 TC-11: AI timeout and fallback behavior

- Create a positive test for AI timeout and fallback behavior.
- Create a negative or unauthorized test for AI timeout and fallback behavior.
- Verify persisted data and side effects after AI timeout and fallback behavior.
- Record the expected API status, UI state, audit event, and notification behavior.

17.18.12 TC-12: AI structured-output validation

- Create a positive test for AI structured-output validation.
- Create a negative or unauthorized test for AI structured-output validation.
- Verify persisted data and side effects after AI structured-output validation.
- Record the expected API status, UI state, audit event, and notification behavior.

17.18.13 TC-13: audit event creation

- Create a positive test for audit event creation.
- Create a negative or unauthorized test for audit event creation.
- Verify persisted data and side effects after audit event creation.
- Record the expected API status, UI state, audit event, and notification behavior.

17.18.14 TC-14: privacy redaction

- Create a positive test for privacy redaction.
- Create a negative or unauthorized test for privacy redaction.
- Verify persisted data and side effects after privacy redaction.
- Record the expected API status, UI state, audit event, and notification behavior.

17.18.15 TC-15: accessibility keyboard flow

- Create a positive test for accessibility keyboard flow.
- Create a negative or unauthorized test for accessibility keyboard flow.
- Verify persisted data and side effects after accessibility keyboard flow.
- Record the expected API status, UI state, audit event, and notification behavior.

17.18.16 TC-16: responsive layout

- Create a positive test for responsive layout.
- Create a negative or unauthorized test for responsive layout.
- Verify persisted data and side effects after responsive layout.
- Record the expected API status, UI state, audit event, and notification behavior.

17.18.17 TC-17: empty and loading states

- Create a positive test for empty and loading states.
- Create a negative or unauthorized test for empty and loading states.

- Verify persisted data and side effects after empty and loading states.
- Record the expected API status, UI state, audit event, and notification behavior.

17.18.18 TC-18: error recovery

- Create a positive test for error recovery.
- Create a negative or unauthorized test for error recovery.
- Verify persisted data and side effects after error recovery.
- Record the expected API status, UI state, audit event, and notification behavior.

17.18.19 TC-19: database index usage

- Create a positive test for database index usage.
- Create a negative or unauthorized test for database index usage.
- Verify persisted data and side effects after database index usage.
- Record the expected API status, UI state, audit event, and notification behavior.

17.18.20 TC-20: backup restoration

- Create a positive test for backup restoration.
- Create a negative or unauthorized test for backup restoration.
- Verify persisted data and side effects after backup restoration.
- Record the expected API status, UI state, audit event, and notification behavior.

17.18.21 Required Test Layers

- Unit tests for scoring, permissions, lifecycle transitions, and formatting utilities.
- Integration tests for repositories, API handlers, queues, storage, and external-service adapters.
- End-to-end tests for the primary success flow and one important failure flow.
- Contract tests for AI structured output and third-party webhook payloads.
- Accessibility checks using automated tooling plus keyboard review.
- Load tests for dashboard reads, search, and one common write endpoint.
- Backup restoration and deployment rollback rehearsal.

17.19 Detailed Product Backlog

17.19.1 US-01: account access

Story: As a Student, I want to manage account access so that I can complete my responsibility with clear status and evidence. **Acceptance criteria:**

1. The account access entry point is visible only to authorized roles.
2. Required account access fields have labels, help text, validation, and accessible errors.
3. A successful account access action persists data and displays a confirmation.
4. A failed account access action preserves user input when safe and explains recovery.
5. The backend rejects unauthorized or malformed account access requests.
6. Important account access changes appear in activity or audit history.

Definition of done:

- Implementation reviewed by another team member.
- Unit or integration tests added.
- Responsive and keyboard behavior checked.
- API documentation and sample data updated.
- No secrets, personal test data, or debug logs committed.

17.19.2 US-02: profile completion

Story: As a Career Mentor, I want to manage profile completion so that I can complete my responsibility with clear status and evidence. **Acceptance criteria:**

1. The profile completion entry point is visible only to authorized roles.
2. Required profile completion fields have labels, help text, validation, and accessible errors.
3. A successful profile completion action persists data and displays a confirmation.
4. A failed profile completion action preserves user input when safe and explains recovery.
5. The backend rejects unauthorized or malformed profile completion requests.
6. Important profile completion changes appear in activity or audit history.

Definition of done:

- Implementation reviewed by another team member.
- Unit or integration tests added.
- Responsive and keyboard behavior checked.
- API documentation and sample data updated.

- No secrets, personal test data, or debug logs committed.

17.19.3 US-03: goal selection

Story: As a Placement Officer, I want to manage goal selection so that I can complete my responsibility with clear status and evidence. **Acceptance criteria:**

1. The goal selection entry point is visible only to authorized roles.
2. Required goal selection fields have labels, help text, validation, and accessible errors.
3. A successful goal selection action persists data and displays a confirmation.
4. A failed goal selection action preserves user input when safe and explains recovery.
5. The backend rejects unauthorized or malformed goal selection requests.
6. Important goal selection changes appear in activity or audit history.

Definition of done:

- Implementation reviewed by another team member.
- Unit or integration tests added.
- Responsive and keyboard behavior checked.
- API documentation and sample data updated.
- No secrets, personal test data, or debug logs committed.

17.19.4 US-04: skill inventory

Story: As a Career Content Administrator, I want to manage skill inventory so that I can complete my responsibility with clear status and evidence. **Acceptance criteria:**

1. The skill inventory entry point is visible only to authorized roles.
2. Required skill inventory fields have labels, help text, validation, and accessible errors.
3. A successful skill inventory action persists data and displays a confirmation.
4. A failed skill inventory action preserves user input when safe and explains recovery.
5. The backend rejects unauthorized or malformed skill inventory requests.
6. Important skill inventory changes appear in activity or audit history.

Definition of done:

- Implementation reviewed by another team member.
- Unit or integration tests added.
- Responsive and keyboard behavior checked.

- API documentation and sample data updated.
- No secrets, personal test data, or debug logs committed.

17.19.5 US-05: gap analysis

Story: As a Student, I want to manage gap analysis so that I can complete my responsibility with clear status and evidence. **Acceptance criteria:**

1. The gap analysis entry point is visible only to authorized roles.
2. Required gap analysis fields have labels, help text, validation, and accessible errors.
3. A successful gap analysis action persists data and displays a confirmation.
4. A failed gap analysis action preserves user input when safe and explains recovery.
5. The backend rejects unauthorized or malformed gap analysis requests.
6. Important gap analysis changes appear in activity or audit history.

Definition of done:

- Implementation reviewed by another team member.
- Unit or integration tests added.
- Responsive and keyboard behavior checked.
- API documentation and sample data updated.
- No secrets, personal test data, or debug logs committed.

17.19.6 US-06: roadmap builder

Story: As a Career Mentor, I want to manage roadmap builder so that I can complete my responsibility with clear status and evidence. **Acceptance criteria:**

1. The roadmap builder entry point is visible only to authorized roles.
2. Required roadmap builder fields have labels, help text, validation, and accessible errors.
3. A successful roadmap builder action persists data and displays a confirmation.
4. A failed roadmap builder action preserves user input when safe and explains recovery.
5. The backend rejects unauthorized or malformed roadmap builder requests.
6. Important roadmap builder changes appear in activity or audit history.

Definition of done:

- Implementation reviewed by another team member.
- Unit or integration tests added.

- Responsive and keyboard behavior checked.
- API documentation and sample data updated.
- No secrets, personal test data, or debug logs committed.

17.19.7 US-07: progress tracking

Story: As a Placement Officer, I want to manage progress tracking so that I can complete my responsibility with clear status and evidence. **Acceptance criteria:**

1. The progress tracking entry point is visible only to authorized roles.
2. Required progress tracking fields have labels, help text, validation, and accessible errors.
3. A successful progress tracking action persists data and displays a confirmation.
4. A failed progress tracking action preserves user input when safe and explains recovery.
5. The backend rejects unauthorized or malformed progress tracking requests.
6. Important progress tracking changes appear in activity or audit history.

Definition of done:

- Implementation reviewed by another team member.
- Unit or integration tests added.
- Responsive and keyboard behavior checked.
- API documentation and sample data updated.
- No secrets, personal test data, or debug logs committed.

17.19.8 US-08: project planner

Story: As a Career Content Administrator, I want to manage project planner so that I can complete my responsibility with clear status and evidence. **Acceptance criteria:**

1. The project planner entry point is visible only to authorized roles.
2. Required project planner fields have labels, help text, validation, and accessible errors.
3. A successful project planner action persists data and displays a confirmation.
4. A failed project planner action preserves user input when safe and explains recovery.
5. The backend rejects unauthorized or malformed project planner requests.
6. Important project planner changes appear in activity or audit history.

Definition of done:

- Implementation reviewed by another team member.

- Unit or integration tests added.
- Responsive and keyboard behavior checked.
- API documentation and sample data updated.
- No secrets, personal test data, or debug logs committed.

17.19.9 US-09: notifications

Story: As a Student, I want to manage notifications so that I can complete my responsibility with clear status and evidence. **Acceptance criteria:**

1. The notifications entry point is visible only to authorized roles.
2. Required notifications fields have labels, help text, validation, and accessible errors.
3. A successful notifications action persists data and displays a confirmation.
4. A failed notifications action preserves user input when safe and explains recovery.
5. The backend rejects unauthorized or malformed notifications requests.
6. Important notifications changes appear in activity or audit history.

Definition of done:

- Implementation reviewed by another team member.
- Unit or integration tests added.
- Responsive and keyboard behavior checked.
- API documentation and sample data updated.
- No secrets, personal test data, or debug logs committed.

17.19.10 US-10: search and filters

Story: As a Career Mentor, I want to manage search and filters so that I can complete my responsibility with clear status and evidence. **Acceptance criteria:**

1. The search and filters entry point is visible only to authorized roles.
2. Required search and filters fields have labels, help text, validation, and accessible errors.
3. A successful search and filters action persists data and displays a confirmation.
4. A failed search and filters action preserves user input when safe and explains recovery.
5. The backend rejects unauthorized or malformed search and filters requests.
6. Important search and filters changes appear in activity or audit history.

Definition of done:

- Implementation reviewed by another team member.
- Unit or integration tests added.
- Responsive and keyboard behavior checked.
- API documentation and sample data updated.
- No secrets, personal test data, or debug logs committed.

17.19.11 US-11: reports

Story: As a Placement Officer, I want to manage reports so that I can complete my responsibility with clear status and evidence. **Acceptance criteria:**

1. The reports entry point is visible only to authorized roles.
2. Required reports fields have labels, help text, validation, and accessible errors.
3. A successful reports action persists data and displays a confirmation.
4. A failed reports action preserves user input when safe and explains recovery.
5. The backend rejects unauthorized or malformed reports requests.
6. Important reports changes appear in activity or audit history.

Definition of done:

- Implementation reviewed by another team member.
- Unit or integration tests added.
- Responsive and keyboard behavior checked.
- API documentation and sample data updated.
- No secrets, personal test data, or debug logs committed.

17.19.12 US-12: administration

Story: As a Career Content Administrator, I want to manage administration so that I can complete my responsibility with clear status and evidence. **Acceptance criteria:**

1. The administration entry point is visible only to authorized roles.
2. Required administration fields have labels, help text, validation, and accessible errors.
3. A successful administration action persists data and displays a confirmation.
4. A failed administration action preserves user input when safe and explains recovery.
5. The backend rejects unauthorized or malformed administration requests.
6. Important administration changes appear in activity or audit history.

Definition of done:

- Implementation reviewed by another team member.
- Unit or integration tests added.
- Responsive and keyboard behavior checked.
- API documentation and sample data updated.
- No secrets, personal test data, or debug logs committed.

17.20 Implementation Roadmap

17.20.1 Phase 0: Discovery

Define users, assumptions, success measures, constraints, and out-of-scope behavior. **Exit evidence:**

- A reviewed artifact for phase 0: discovery is stored in the project documentation.
- At least one demonstrable AI Career Roadmap Generator workflow is improved or de-risked.
- Open decisions, blockers, and technical debt are recorded with owners.
- The next phase has clear acceptance criteria and test data.

17.20.2 Phase 1: Foundation

Set up repositories, environments, design tokens, routing, API conventions, and database access. **Exit evidence:**

- A reviewed artifact for phase 1: foundation is stored in the project documentation.
- At least one demonstrable AI Career Roadmap Generator workflow is improved or de-risked.
- Open decisions, blockers, and technical debt are recorded with owners.
- The next phase has clear acceptance criteria and test data.

17.20.3 Phase 2: Identity

Implement authentication, authorization, profile completion, and account security. **Exit evidence:**

- A reviewed artifact for phase 2: identity is stored in the project documentation.
- At least one demonstrable AI Career Roadmap Generator workflow is improved or de-risked.
- Open decisions, blockers, and technical debt are recorded with owners.
- The next phase has clear acceptance criteria and test data.

17.20.4 Phase 3: Core Records

Build the primary create, read, update, archive, and search workflows. **Exit evidence:**

- A reviewed artifact for phase 3: core records is stored in the project documentation.
- At least one demonstrable AI Career Roadmap Generator workflow is improved or de-risked.
- Open decisions, blockers, and technical debt are recorded with owners.
- The next phase has clear acceptance criteria and test data.

17.20.5 Phase 4: Collaboration

Add requests, reviews, comments, notifications, and role-specific work queues. **Exit evidence:**

- A reviewed artifact for phase 4: collaboration is stored in the project documentation.
- At least one demonstrable AI Career Roadmap Generator workflow is improved or de-risked.
- Open decisions, blockers, and technical debt are recorded with owners.
- The next phase has clear acceptance criteria and test data.

17.20.6 Phase 5: Intelligence

Implement deterministic scoring first, then add AI assistance behind a stable service boundary. **Exit evidence:**

- A reviewed artifact for phase 5: intelligence is stored in the project documentation.
- At least one demonstrable AI Career Roadmap Generator workflow is improved or de-risked.
- Open decisions, blockers, and technical debt are recorded with owners.
- The next phase has clear acceptance criteria and test data.

AI Gate

> Do not move the AI feature into the primary user flow until its evaluation results, failure behavior, privacy handling, and fallback are documented.

17.20.7 Phase 6: Analytics

Create operational dashboards, event tracking, reports, and export workflows. **Exit evidence:**

- A reviewed artifact for phase 6: analytics is stored in the project documentation.
- At least one demonstrable AI Career Roadmap Generator workflow is improved or de-risked.
- Open decisions, blockers, and technical debt are recorded with owners.
- The next phase has clear acceptance criteria and test data.

17.20.8 Phase 7: Hardening

Complete threat review, accessibility checks, performance testing, and recovery procedures. **Exit evidence:**

- A reviewed artifact for phase 7: hardening is stored in the project documentation.
- At least one demonstrable AI Career Roadmap Generator workflow is improved or de-risked.
- Open decisions, blockers, and technical debt are recorded with owners.
- The next phase has clear acceptance criteria and test data.

17.20.9 Phase 8: Pilot

Release to a controlled cohort, measure behavior, collect feedback, and correct product assumptions. **Exit evidence:**

- A reviewed artifact for phase 8: pilot is stored in the project documentation.
- At least one demonstrable AI Career Roadmap Generator workflow is improved or de-risked.
- Open decisions, blockers, and technical debt are recorded with owners.
- The next phase has clear acceptance criteria and test data.

17.20.10 Phase 9: Production

Deploy with monitoring, support ownership, release notes, and a rollback strategy. **Exit evidence:**

- A reviewed artifact for phase 9: production is stored in the project documentation.
- At least one demonstrable AI Career Roadmap Generator workflow is improved or de-risked.
- Open decisions, blockers, and technical debt are recorded with owners.
- The next phase has clear acceptance criteria and test data.

17.21 Semester Execution Plan

Week	Primary Outcome
Week 1	Problem research, stakeholder interviews, and baseline measurement
Week 2	Personas, journey map, scope, risks, and success metrics
Week 3	Wireframes, design system, domain model, and API contract
Week 4	Project setup, authentication, authorization, and CI checks
Week 5	Implement Goal Selection and Skill Inventory
Week 6	Implement Gap Analysis and Roadmap Builder
Week 7	Implement Progress Tracking and notifications
Week 8	Complete role dashboards, search, filters, and audit history

Week	Primary Outcome
Week 9	Build and evaluate career pathway generation
Week 10	Integrate AI fallback, analytics, reports, and exports
Week 11	Security review, accessibility checks, performance testing, and bug fixing
Week 12	Pilot deployment, user testing, documentation, presentation, and final demo rehearsal

17.22 Deployment and Operations

1. Create separate development, test, staging, and production configuration.
2. Build reproducible frontend and backend artifacts.
3. Run database migrations through a versioned deployment step.
4. Use managed HTTPS, database backups, private storage, and environment secrets.
5. Add health, readiness, and dependency checks.
6. Collect structured application logs with request identifiers.
7. Monitor API errors, latency, job failures, AI cost, and storage usage.
8. Seed demonstration data without using real sensitive student records.
9. Document rollback steps for application and database changes.
10. Assign an owner for production incidents and user support.

17.22.1 Environment Variables

Listing 17.6: Illustrative environment configuration

```
NODE_ENV=production
PORT=5000
DATABASE_URL=managed-database-connection
REDIS_URL=managed-redis-connection
SESSION_SECRET=secret-manager-reference
OBJECT_STORAGE_BUCKET=private-bucket-name
AI_PROVIDER_API_KEY=secret-manager-reference
AI_MODEL=approved-model-name
APP_ORIGIN=https://student-project.example
LOG_LEVEL=info
```

17.23 Documentation Deliverables

- Problem statement and evidence from user research
- Personas and user journey

- Scope, assumptions, and out-of-scope list
- Functional and non-functional requirements
- System context and component diagrams
- Entity relationship or document model diagram
- API specification with example requests and errors
- Security, privacy, and risk register
- AI model card or AI feature note
- Test plan and execution evidence
- Deployment guide and environment variable reference
- User manual and administrator manual
- Known limitations and future roadmap

17.24 Demonstration Script

1. Introduce the real problem in one minute: Students consume disconnected learning resources without a realistic sequence that reflects their current skills, time, goals, and evidence.
2. Show the Student onboarding and profile experience.
3. Create or submit a real CareerGoal record.
4. Demonstrate Roadmap Builder with a visible explanation of its logic.
5. Switch to the Career Mentor role and complete the required response or review.
6. Return to the original user and show notification, status, and history.
7. Show an analytics change for roadmap activation.
8. Demonstrate one validation error and one permission boundary.
9. Explain the AI limitation and fallback.
10. Close with measured outcomes, known limitations, and the next product milestone.

17.25 Viva and Interview Preparation

1. What evidence shows that AI Career Roadmap Generator solves a real problem?
2. Why did you choose a modular monolith instead of microservices?
3. How is roadmap activation calculated and validated?
4. Which fields belong to CareerGoal, and why?

5. How does the backend enforce role and ownership permissions?
6. What happens when the AI provider is unavailable or returns invalid data?
7. How did you evaluate AI accuracy, bias, usefulness, and cost?
8. Which database indexes support the most important queries?
9. How do you prevent duplicate writes and inconsistent state?
10. Which data is sensitive, and how can a user correct or delete it?
11. What tests protect the primary workflow?
12. What is the largest known limitation of the current release?

17.26 Evaluation Rubric

Area	Marks and Evidence
Problem understanding	10 marks: research quality, stakeholder clarity, and validated need.
Product design	10 marks: scope, workflows, usability, accessibility, and error-state design.
Frontend engineering	15 marks: responsive implementation, state handling, maintainability, and API integration.
Backend engineering	20 marks: domain rules, authorization, validation, data integrity, and error handling.
Data and AI	15 marks: justified use, evaluation, structured outputs, safety, fallback, and limitations.
Testing and security	15 marks: meaningful automated tests, threat controls, privacy, and evidence.
Deployment and operations	5 marks: reproducible release, monitoring, backup, and rollback awareness.
Documentation and viva	10 marks: diagrams, decisions, demonstration quality, and technical explanation.

17.27 Project Completion Checklist

- [ ] Problem validated with target users
- [ ] MVP and out-of-scope list approved
- [ ] Roles and authorization matrix implemented
- [ ] Primary workflow works end to end
- [ ] Database constraints and indexes reviewed

- [ ] API documentation matches implementation
- [ ] Loading, empty, success, and error states completed
- [ ] AI feature evaluated and fallback tested
- [ ] Security and privacy review completed
- [ ] Accessibility review completed
- [ ] Automated test suite passing
- [ ] Staging deployment verified
- [ ] Backup and rollback steps documented
- [ ] Demo data prepared
- [ ] Final report and viva answers reviewed

17.28 Conclusion

AI Career Roadmap Generator can become a strong professional project when the team treats it as a real product rather than a collection of pages. The strongest submission will demonstrate trustworthy data, clear role workflows, explainable roadmap builder, tested permissions, responsible AI, and measurable outcomes. Students should keep the first release focused, collect pilot evidence, and use that evidence to justify every major extension.

Cross-Project Architecture Checklist

- Can every important record be traced to an owner, actor, status, and timestamp?
- Can the backend reject every action the frontend hides from an unauthorized user?
- Can the main workflow recover from network, storage, queue, and AI failures?
- Can a reviewer understand why a score, match, recommendation, or alert was produced?
- Can the team restore data, roll back a release, and identify the cause of a production error?
- Can users correct inaccurate information and report harmful or abusive activity?
- Can the product be demonstrated without exposing real personal or secret data?

Final Advice to Student Teams

Build the smallest version that proves the full value chain. A complete, secure, understandable workflow is more impressive than twenty unfinished features. Keep weekly evidence: interview notes, diagrams, commits, tests, screenshots, evaluation results, and deployment logs. During the final presentation, explain not only what works but also why it was designed that way, where it can fail, and how the next version would improve.